

RAG App 01

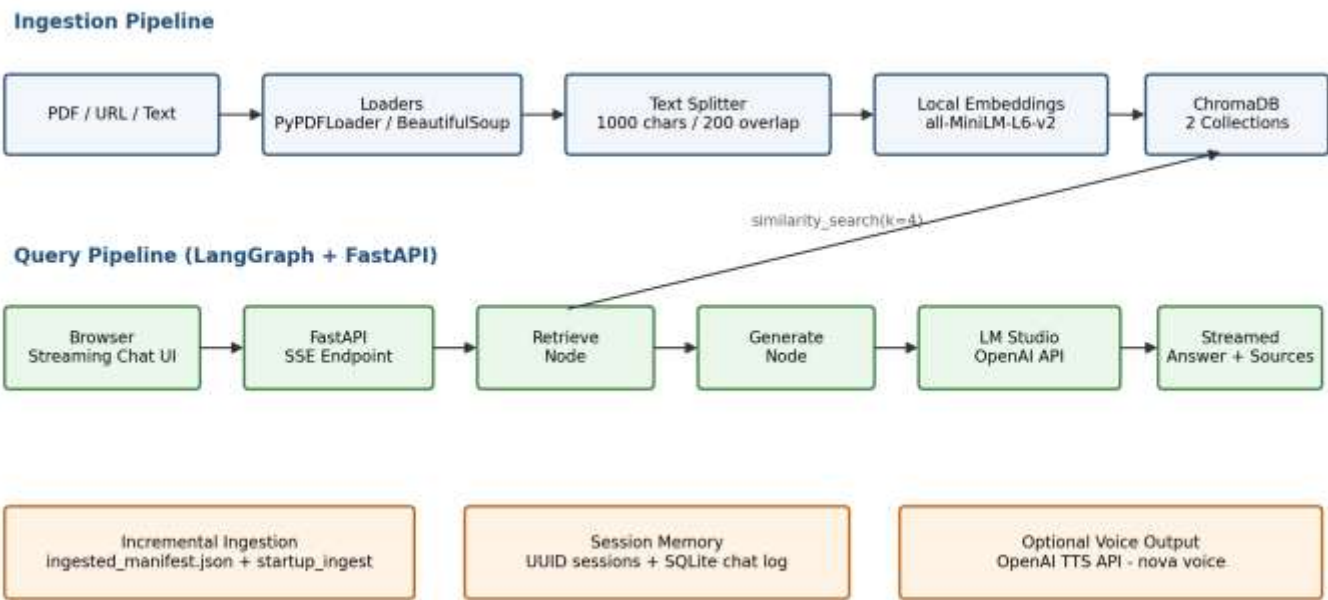
Technical Architecture Document

Local-first Retrieval-Augmented Generation system for domain-specific AI education and senior services content. Documentation provided by Eric Frayer.

For more information: info@cincyseniors.org

Architecture Pattern	Local-first Retrieval-Augmented Generation (RAG)
Primary Runtime	Python 3.12, FastAPI, Uvicorn
LLM Endpoint	LM Studio local OpenAI-compatible API at 192.168.68.69:1234/v1
Vector Store	ChromaDB persistent SQLite backend with two named collections
Document Status	Professional architecture draft based on supplied technical outline

RAG App 01 - System Architecture



1. Overview

RAG App 01 is a local-first Retrieval-Augmented Generation system designed for domain-specific AI education and senior services content. The application combines local document ingestion, local sentence embeddings, persistent vector search, and a locally hosted LLM served through LM Studio. A FastAPI web server exposes the RAG workflow to a browser-based streaming chat interface.

The design emphasizes privacy, controlled content scope, and extensibility. Core content is indexed into ChromaDB, user questions are routed through a LangGraph workflow, and generated responses are streamed to the browser with source citations. A specialized CincySeniors chat endpoint adds multi-turn conversation memory and persistent session logging.

2. Architecture Principles

- **Local-first operation:** Use local LLM inference and local embeddings wherever practical to reduce dependency on hosted model infrastructure.
- **Retrieval-grounded answers:** Retrieve relevant content from curated collections before generation, then return answer text with source citations.
- **Incremental ingestion:** Avoid reprocessing unchanged files by tracking file modification times in an ingestion manifest.
- **Streaming user experience:** Use Server-Sent Events so the browser receives model output token by token.
- **Extensible orchestration:** Model the query pipeline as a LangGraph StateGraph so additional routing, re-ranking, or self-correction nodes can be added later.

3. System Architecture

3.1 Ingestion Pipeline

The ingestion pipeline converts source content into chunked, embedded documents stored in ChromaDB. PDF files, web URLs, and plain text snippets are normalized into LangChain Document objects, split into overlapping chunks, embedded locally, and stored in named vector collections.

```
PDF / URL / Text
-> PyPDFLoader / BeautifulSoup / Direct Document
-> RecursiveCharacterTextSplitter (1000 chars / 200 overlap)
-> HuggingFaceEmbeddings (all-MiniLM-L6-v2, 384 dimensions)
-> ChromaDB persistent vector store (SQLite backend)
```

3.2 Query Pipeline

The query pipeline is orchestrated with LangGraph. A user question enters the retrieve node, which performs vector similarity search against ChromaDB. Retrieved documents are assembled into prompt context and sent to ChatOpenAI configured to use the LM Studio OpenAI-compatible API. The generated answer is streamed back through FastAPI to the browser.

```
User Question
-> [retrieve node] similarity_search(k=4) against ChromaDB
-> [generate node] ChatPromptTemplate + ChatOpenAI
    LM Studio endpoint: 192.168.68.69:1234/v1
-> Streamed Answer + Source Citations
-> FastAPI Server-Sent Events
-> Browser Chat UI
```

4. Content Loading and Collections

RAG App 01 supports multiple source types and routes them into one of two persistent ChromaDB collections. Incremental ingestion is coordinated by the startup_ingest FastAPI startup hook and tracked through ingested_manifest.json.

Source Type	Loader / Method	Target Collection
PDF files	PyPDFLoader from LangChain Community	Default / CincySeniors
Web URLs	BeautifulSoup4 scraper	CincySeniors
Plain text snippets	Direct string to LangChain Document	CincySeniors

4.1 Incremental Ingestion

The application compares file modification times against ingested_manifest.json on each startup. Only new or modified content is processed, which avoids unnecessary re-chunking and re-embedding of stable source documents.

4.2 Vector Collections

Collection	Purpose	Current Scope
Default	General AI education knowledge base	4 general AI education PDFs
cincyseniors	Senior services and older-adult focused knowledge base	22+ items including PDFs, URLs, and text snippets

5. LangChain and LangGraph Design

LangGraph Query Workflow



5.1 StateGraph Workflow

The query pipeline is modeled as a typed LangGraph StateGraph in `app/graph.py`. The graph state contains three fields: question, documents, and answer. The graph is intentionally minimal, with two sequential nodes.

```
START -> retrieve -> generate -> END
```

This pattern is deliberately extensible. Future nodes can be inserted between retrieve and generate to support query routing, document re-ranking, grounding checks, self-correction, or specialized tool calls.

5.2 LCEL Chain Composition

The generate and generate_stream nodes use LangChain Expression Language to compose the prompt and LLM as a Runnable. The blocking path uses `invoke`, while the streaming path uses `stream` to yield tokens incrementally.

```
chain = prompt | llm          # pipe operator builds a Runnable
chain.invoke({...})          # blocking call
chain.stream({...})           # token-by-token generator
```

Component	Package	Role
PyPDFLoader	langchain-community	Converts PDF pages into Document objects with page metadata
RecursiveCharacterTextSplitter	langchain-text-splitters	Creates overlapping text chunks for semantic retrieval
HuggingFaceEmbeddings	langchain-huggingface	Generates local CPU-based sentence embeddings
Chroma	langchain-chroma	Persistent vector store with <code>similarity_search(q, k=4)</code>
ChatOpenAI	langchain-openai	Connects LangChain to LM Studio through an OpenAI-compatible interface
ChatPromptTemplate	langchain-core	Defines system and human message templates with context and question slots
StateGraph	langgraph	Orchestrates the retrieve-to-generate workflow

6. Streaming, Memory, and Logging

6.1 Streaming Responses

The `/api/query/stream` and `/cindy/chat` endpoints use FastAPI StreamingResponse with Server-Sent Events. The `generate_stream` node yields tokens from `chain.stream(...)`, which are forwarded to the browser as token events. A final done event carries response metadata and source citations.

```
{ "type": "token", "token": "..."}
{ "type": "done", "sources": [...]}
```

6.2 Multi-turn Memory for CincySeniors

The /cindy/chat endpoint maintains conversation context by passing the full messages array into the prompt context. Messages are represented using LangChain HumanMessage and AIMessage objects. Sessions are keyed by UUID, and all turns are persisted to SQLite through chat_logger.py.

7. Technology Stack

Layer	Technology
LLM	LM Studio local OpenAI-compatible API
Orchestration	LangGraph 1.0.9
LangChain Framework	LangChain 1.2.10 plus ecosystem packages
Embeddings	all-MiniLM-L6-v2, 384-dimensional local CPU embeddings
Vector Store	ChromaDB 1.5.1 with SQLite backend
Web Server	FastAPI 0.131.0 with Uvicorn
Chat Logging	SQLite database cindy_chat_log.db
Text-to-Speech	OpenAI TTS API using nova voice, optional
Frontend	Vanilla JavaScript, marked.js, highlight.js
Runtime	Python 3.12

8. Operational Considerations

- **Model availability:** LM Studio must be running and serving an OpenAI-compatible endpoint before the RAG server sends generation requests.
- **Embedding reproducibility:** The all-MiniLM-L6-v2 embedding model should remain stable for a given collection. Changing embedding models generally requires re-indexing.
- **Storage management:** ChromaDB and SQLite files should be included in backup or refresh procedures depending on whether logs and indexes are treated as durable assets.
- **Content freshness:** The ingestion manifest detects changed source files by modification time. URL-based content may require additional refresh policy if web content changes without a local file timestamp.
- **Privacy posture:** The core RAG flow runs local inference and local embeddings. Optional TTS uses an external OpenAI API and should be treated as an external service dependency.

9. Extension Opportunities

- **Re-ranking node:** Add a re-ranking step after retrieval to improve source ordering before generation.
- **Grounding validator:** Add a self-check node to compare generated answers against retrieved context before streaming completion.
- **Collection routing:** Route questions to Default, CincySeniors, or multiple collections based on intent classification.
- **Admin ingestion UI:** Expose upload, URL ingestion, re-indexing, and manifest status through an authenticated admin interface.
- **Observability dashboard:** Track retrieval scores, token throughput, endpoint latency, and source usage over time.

Appendix A. Supplied Architecture Outline Mapping

The supplied outline was reorganized into the following architecture sections: overview, system architecture, content loading, LangChain/LangGraph design, streaming and memory, technology stack, operational considerations, and extension opportunities. Technical names, versions, endpoints, collections, and processing parameters were preserved.